On the Lovasz Local Lemma, Combinatorial Proofs, and Moser's Entropic Argument

CS-2446-1

Sivasanjai G A

Term Paper

Department Of Computer Science

Ashoka University

India

Motivation and Background:

Lovász Local Lemma (LLL) is a powerful mathematical tool, and a cornerstone of the probabilistic method. The probabilistic method was developed in the 70s by Hungarian mathematicians László Lovász and Paul Erdős to guarantee the existence of certain combinatorial structures[1]. Later, the theoretical computer science community realized its immense usefulness as a tool to prove results about the satisfiability problem, graph colorings, tree codes, integer programming, etc. This kicked off a decades long quest to constructify the LLL culminating in the breakthrough work by Robin A. Moser and Gabor Tardos, who, in 2009, proved the effectiveness of the simplest non-trivial randomized search algorithm, resulting in a constructive LLL of utmost simplicity [2,3]. In the same year, the prominent contemporary mathematician Terrance Tao, in his blog, followed up on the aforementioned work with his presentation of the analysis of combinatorial construction algorithms in terms of the exploitation of different types of bounded and monotonically increasing properties[4]; he also first mentions the idea of how entropy– one such property– is of use to analyse the Moser-Tardos algorithm. Soon, Moser follows up his original proof with a much more elegant analysis in terms of entropy compression. In the following sections, these exciting developments are systemically examined and presented. First, the background necessary is presented; the notation format used by Sloan Nietert in his blog[5] is adapted for consistency.

The Probablistic Method

The probablistic method is a widely used mathematical tool first fashioned by Paul Erdős in the 1940s to prove some results in Ramsey theory. The set up usually involves a large and finite search space Ω and a set β of flaws that are to be avoided. Each $B \in \beta$ is a subset of Ω whose objects share some undesirable trait. Hence, the search is for a flawless object $\sigma \in \Omega \bigcup_{B \in \beta} B$.

The probabilistic approach assigns a probability measure to Ω, interpreting each $B \in \mathcal{B}$ as a "bad" event. Leveraging tools from probability theory, one aims to prove that the probability of none of the bad events occurring,

$$\Pr\left[\bigwedge_{B \in \mathcal{B}} \overline{B}\right] = 1 - \Pr\left[\bigvee_{B \in \mathcal{B}} B\right],$$

is strictly positive. This positive probability certifies the existence of a flawless object within Ω. Although this approach often mirrors a counting argument, it has been remarkably effective, enabling combinatorialists to circumvent the need to redevelop established probabilistic techniques. Classical applications of the probabilistic method include problems in graph coloring, while the k-SAT problem is particularly pertinent to computer science.

Consider a Boolean formula ϕ in conjunctive normal form (CNF) with n variables, m clauses, and exactly k distinct variables in each clause. The k-SAT decision problem involves determining whether there exists an assignment of truth values to the variables that satisfies all clauses in ϕ. For the purposes of this analysis, we adopt a slight variation of the traditional k-SAT definition by requiring that the variables within each clause are distinct. In this context, the search space is defined as

$$\Omega = \{\text{True}, \text{False}\}^n,$$

where each element represents a potential assignment of truth values to the n variables. The set of flaws is defined as

$$\mathcal{B}_i = \{\sigma \in \Omega \mid \sigma \text{ violates clause } i \text{ of } \phi\},$$

for $i = 1, \ldots, m$. We employ the uniform probability measure over Ω, whereby each variable is independently assigned True or False with equal probability. Under this measure, the probability of violating any particular clause $\mathcal{B}_i$ is

$$\Pr[\mathcal{B}_i] = \frac{1}{2^k},$$

since there exists exactly one assignment to the k distinct variables in a clause that renders it false.

A fundamental tool in the probabilistic method is the union bound. Applied to the k-SAT problem, the union bound ensures the existence of a satisfying assignment provided that the number of clauses is sufficiently small. Specifically, if

$$m < 2^k,$$

then the probability of finding an assignment that satisfies all clauses is positive. Formally,

$$\begin{aligned}
\Pr\left[\bigwedge_{i=1}^{m} \overline{\mathcal{B}_i}\right] &= 1 - \Pr\left[\bigvee_{i=1}^{m} \mathcal{B}_i\right] \\
&\geq 1 - \sum_{i=1}^{m} \Pr[\mathcal{B}_i] \\
&= 1 - \frac{m}{2^k} \\
&> 0,
\end{aligned}$$

provided that $m < 2^k$. This inequality confirms the existence of a satisfying assignment under the given condition[6].

An alternative scenario arises when all clauses are mutually disjoint, meaning that the events $\mathcal{B}_i$ are independent. In this case, the probability of avoiding all bad events is given by

$$\Pr\left[\bigwedge_{i=1}^{m} \overline{\mathcal{B}_i}\right] = \prod_{i=1}^{m} \Pr\left[\overline{\mathcal{B}_i}\right] = \left(1 - \frac{1}{2^k}\right)^m,$$

which is positive as long as m is bounded appropriately.

In general, when each bad event occurs with probability at most p, the union bound provides a sufficient condition for the existence of a flawless object. The relationship between different methods, their assumptions, and the corresponding sufficient conditions can be summarized in the following table from Sloan Nietert's blog[5]:

Table 1: Comparison of Methods for Establishing the Existence of a Flawless Object

Method	**Assumptions on ...**	**Sufficient Condition for Flawless Object**
Union Bound	No specific assumptions	$p < \frac{1}{n}$
Independence	Events are independent	$p < 1$
Local Dependencies	??	$p <?$

As depicted in Table 1 [n], it is the natural next step to ask whether it is possible to interpolate between the aforementioned results with some assumptions on local event structure.

The Lovász Local Lemma:

The answer to the previously posted question was given by Lászlo Lovász in the 1970s upon his coining of the local lemma to prove some results on hypergrah colorability[1]. LLL has two different versions: symmetric and asymmetric. The symmetric version is usually applied when the bad events $B \in \beta$ are identical upto some measure preserving permutation of the objects.

[Symmetric Lovász Local Lemma] Let $\mathcal{B}$ be a set of events such that each event occurs with probability at most p and is mutually independent of all but at most d other events. If

$$e \cdot p \cdot (d+1) \leq 1,$$

then the probability that none of the events in $\mathcal{B}$ occur is positive, i.e.,

$$\Pr\left[\bigwedge_{B \in \mathcal{B}} \overline{B}\right] > 0.$$

For dependency bounds satisfying $d \geq 1$, Shearer later refined this condition, demonstrating that the requirement can be relaxed to

$$e \cdot p \cdot d \leq 1.$$

[n] The dependency bound d effectively characterizes the local structure of event dependencies.

Application to k-SAT

To highlight the implications of the Lovász Local Lemma in computational problems, consider its application to the k-SAT problem.

Let ϕ be a Boolean formula in conjunctive normal form (CNF) with n variables, m clauses, and exactly k distinct variables per clause. Suppose that each variable in ϕ appears in at most $\frac{2^k}{ke}$ clauses. Consequently, each clause can share variables with at most

$$d = k \cdot \frac{2^k}{ke} = \frac{2^k}{e}$$

other clauses. This establishes a dependency bound of $d = \frac{2^k}{e}$ among the bad events, where each bad event corresponds to the violation of a clause.

The probability of any single bad event $\mathcal{B}_i$ occurring is

$$p = 2^{-k},$$

since exactly one of the 2^k possible assignments to the k variables in a clause renders it false. Substituting these values into Shearer's condition yields

$$e \cdot p \cdot d = e \cdot 2^{-k} \cdot \frac{2^k}{e} = 1.$$

This calculation satisfies the relaxed Lovász Local Lemma condition, thereby guaranteeing the existence of a satisfying assignment for ϕ.

Dependency Graph

[Dependency Graph] Let $\mathcal{E}$ be a set of events within a probability space. A graph G with vertex set $\mathcal{E}$ is termed a *dependency graph* for $\mathcal{E}$ if each event is mutually independent of all other events except those corresponding to its adjacent vertices in G.

In the context of k-SAT, there exists a unique edge-minimum dependency graph where edges connect events corresponding to clauses that share variables. However, for general event sets, no canonical dependency graph is defined. Additionally, transitioning from an event set to a dependency graph results in the loss of specific information about the events themselves.

Asymmetric Lovász Local Lemma

In its asymmetric form, LLL can accommodate varying probabilities of bad events and differing dependency structures. The

[Asymmetric Lovász Local Lemma] Let $\mathcal{B}$ be a set of bad events with dependency graph G. Suppose there exists a vector $\mathbf{x} = \{x_B\}_{B \in \mathcal{B}}$ with each $x_B \in (0, 1)$ such that for every $B \in \mathcal{B}$,

the following condition holds:

$$\Pr[B] \leq x_B \prod_{C \in \mathcal{N}_G(B)} (1 - x_C),$$

where $\mathcal{N}_G(B)$ denotes the set of neighbors of B in the graph G. Under these circumstances, the probability that none of the bad events occur is positive, i.e.,

$$\Pr\left[\bigwedge_{B \in \mathcal{B}} \overline{B}\right] > 0.$$

The symmetric LLL can be derived as a corollary of the asymmetric version by setting each $x_B = \frac{1}{d+1}$, where d is the maximum degree in the dependency graph G.

$$\Pr[B] \leq \frac{1}{d+1} \prod_{C \in \mathcal{N}_G(B)} \left(1 - \frac{1}{d+1}\right).$$

Since $\prod_{C \in \mathcal{N}_G(B)} \left(1 - \frac{1}{d+1}\right) \geq \left(1 - \frac{1}{d+1}\right)^d \geq \frac{1}{e}$ for sufficiently large d, the condition simplifies to:

$$e \cdot p \cdot (d+1) \leq 1,$$

where p is the maximum probability of any bad event. This aligns with the symmetric LLL condition, ensuring that the probability of avoiding all bad events remains positive.

Moser's Algorithm:

The necessary background in place, it is now possible to state and examine Moser's algorithm. One can begin by noticing that in most applications of LLL, the bad events of interest are generated by a finite set of independent random variables. Capturing this structure allows for a nice approach to construct LLL. Consider the following definitions:

Event-Variable Bigraph

[Event-Variable Bigraph] Let $\mathcal{B}$ be a set of events and $\mathcal{X}$ be a set of mutually independent random variables such that each event $B \in \mathcal{B}$ is completely determined by a subset of variables

$\mathrm{vbl}(B) \subseteq \mathcal{X}$. The *event-variable bigraph* of this system is defined as the bipartite graph $H = (\mathcal{B} \cup \mathcal{X}, E)$, where an edge $(B, X) \in E$ exists if and only if $X \in \mathrm{vbl}(B)$.

Base Dependency Graph

[Base Dependency Graph] Given an event-variable bigraph $H = (\mathcal{B} \cup \mathcal{X}, E)$, the *base dependency graph* $G_H = (\mathcal{B}, E')$ is defined such that an edge $(B, B') \in E'$ exists if and only if $\mathrm{vbl}(B) \cap \mathrm{vbl}(B') \neq \emptyset$.

In the context of the k-SAT problem, the base dependency graph G_H corresponds to the unique edge-minimum dependency graph where edges connect events (clauses) that share one or more variables. However, for general sets of events, there is no canonical choice for the dependency graph.

The base dependency graph G_H serves as a dependency graph for the set of events $\mathcal{B}$. Specifically, G_H is the edge-minimum dependency graph provided that no variable set is unnecessarily large.

The Resampling Algorithm:

Unlike the previous algorithmic approaches which required significant assumptions on top of the above presented variable structure, that of Moser and Tardos is rather simplistic. Given a set β of bad events generated by a finite set of random variables χ, Moser's algorithm finds an assignment of the random variables such that all of the bad events are avoided.

Algorithm 1 Resample

Require: Finite event-variable bigraph $H = (\mathcal{B} \cup \mathcal{X}, E)$, procedure for sampling from each random variable $X \in \mathcal{X}$

Ensure: Assignments $v = (v_X)_{X \in \mathcal{X}}$ of variables such that no event $B \in \mathcal{B}$ occurs

```
Initialize: Randomly sample v_X ~ X for each variable X ∈ 𝒳
while there exists an event B ∈ ℬ that occurs under assignments v do
    Resample v_X ~ X for each variable X ∈ vbl(B)
end while
return v
```

Initally, Moser and Tardos proved the following theorem for running time bounding under asymmetric LLL conditions.

[Running Time of Resample] Let $\mathcal{B}$ be a set of bad events and $\mathcal{X}$ a set of mutually independent random variables, forming an event-variable bigraph $H = (\mathcal{B} \cup \mathcal{X}, E)$. Suppose there exists a vector $\mathbf{x} = \{x_B\}_{B \in \mathcal{B}}$ with each $x_B \in (0, 1)$ such that for every $B \in \mathcal{B}$, the following condition holds:

$$\Pr[B] \leq x_B \prod_{\substack{C \in \mathcal{B} \\ \mathrm{vbl}(B) \cap \mathrm{vbl}(C) \neq \emptyset}} (1 - x_C) \,.$$

Under these circumstances, the Resample algorithm terminates with an assignment of variables $v = (v_X)_{X \in \mathcal{X}}$ such that no event $B \in \mathcal{B}$ occurs, within an expected number of resampling steps bounded by

$$\sum_{B \in \mathcal{B}} \frac{x_B}{1 - x_B}.$$

While the proof of the above theorem is of great interest in its own regard, of interest to this report is the entropy based approach first suggested by Terry Tao in his 2009 blog post (again, inspired by a talk given by Moser himself) and the analysis further presented by Moser.

Constructive Proofs of Combinatorial Objects:

In the frequently referenced blog post, Terence Tao presents a generalization of constructive proofs of combinatorial objects. He posits that such scenarios generally involve some iteration algorithm which continually improves some initial object, say O; each loop of this algorithm replaces O with some better version of itself O' until some desired property of O is attained and the algorithm halts. Clearly, for such an argument to hold, it is necessary that the algorithm used halts in a finite/bounded amount of time.

He then generalises the usual termination strategies of such algorithms by stating that they often exploit a "monotonicity property", i.e, by showing that some key quantity keeps increasing or decreasing with each loop of the algorithm, while staying bounded. Such algorithms can generally be bucketed into having exploited 4 different properties:

Mass Increment Argument

The mass increment argument is a widely recognized method to guarantee termination. In this approach, each improved object A' is made "heavier" than the previous object A by a non-trivial amount. For example, one might ensure that the cardinality of A' is strictly greater than that of A, such that $|A'| \geq |A| + 1$. Alternatively, the mass remaining "outside" of A may be forced to decrease in some defined sense. Provided there exists a substantial upper bound on the mass of A that remains relatively constant throughout the iterative process, and a lower bound on the mass increment at each stage, termination is assured.

This strategy is characteristic of many greedy algorithms, and is also employed in mathematics, for example, the proof of the Hahn decomposition theorem in measure theory. The general methodology involves continuously identifying and incorporating beneficial segments of mass from outside A to form A', thereby utilizing the additive properties of mass. The process terminates when no further usable mass is available for addition, indicating that A is maximal in an L^1 sense and thus possesses the desired properties.

Density Increment Argument

The density increment argument is a variant of the mass increment approach, wherein the focus is on increasing the "density" of A rather than its mass. Consider an ambient space P within which A resides. The objective is to iteratively enhance A to A' and correspondingly adjust P to P' such that the density of A' within P' improves relative to that of A within P, for instance, $|A'|/|P'| \geq |A|/|P| + c$ for some constant $c > 0$. Given that the density of A is inherently bounded above by 1, establishing a sufficient lower bound on the density increment at each iteration allows for an upper bound on the number of algorithmic iterations.

A quintessential example of this technique is Roth's proof demonstrating that every set of integers with positive upper density contains an arithmetic progression of length three. The overarching strategy involves identifying significant density fluctuations within A and subsequently "zooming in" on regions of increased density by appropriately refining A and P. The process concludes when no further substantial density fluctuations are detectable, implying that A is uniformly distributed and thereby satisfies the requisite properties.

Energy Increment Argument

The energy increment argument serves as an L^2 analogue to the L^1-based mass increment and the L^∞-based density increment arguments. In this framework, the aim is to increment the "energy" that A captures from a reference object X, or equivalently, to decrease the energy of X that remains orthogonal to A. Here, A and X are elements of a Hilbert space, and energy is quantified via the norm associated with that space.

A fundamental example is the standard proof of the Szemerédi regularity lemma, where the "energy" is often referred to as the "index." This illustrate how the energy increment strategy exploits square-additivity properties, such as Pythagoras' theorem, to iteratively augment A by incorporating orthogonal energy components from X. The procedure terminates when no further usable energy remains outside of A, signifying that A is maximal in an L^2 sense and thereby satisfies the desired conditions.

Rank Reduction Argument

The rank reduction argument focuses on decreasing the "rank," "dimension," or "order" of an object with each iterative step. A classic example of this approach is the proof in linear algebra that any finite set of vectors contains a linearly independent subset that spans the same subspace. The general strategy involves identifying "collisions" or "dependencies" within A and utilizing these to collapse A into a new object A' with lower rank. The iterative process continues until no further collisions or dependencies exist within A, thereby ensuring that A possesses the desired maximal properties.

Moser Introduction of Entropy Argument:

Tao then goes on to state how Moser might have introduced a new argumentation method involving the compression of entropy which only applies to probablistic algorithms which require a collection of random bits R and other random choices as a part of input.

Algorithmic Framework

Consider a probabilistic algorithm that operates in discrete iterations or loops. In each iteration, the algorithm performs the following steps:

1. **Input:** The algorithm receives an object A and a subset of the random string $\mathcal{R}$.

2. **Transformation:** Utilizing a deterministic procedure, the algorithm transforms A and a portion of $\mathcal{R}$ to produce an improved object A' and a truncated random string $\mathcal{R}'$. This truncation involves discarding the bits of $\mathcal{R}$ that were utilized in the current iteration.

3. **History Logging:** The algorithm maintains a history log H', which records the cumulative sequence of operations and transformations up to the current iteration. This log is crucial for ensuring the partial reversibility of the algorithm.

Partial Reversibility and Information Compression

A critical aspect of the entropy compression argument is the partial reversibility of the algorithm. Specifically, given the improved object A', the truncated random string $\mathcal{R}'$, and the history log H', it is possible to reconstruct the original object A along with the unused portion of the random string $\mathcal{R}$. This reversibility ensures that the transformation from $(A, \mathcal{R})$ to $(A', \mathcal{R}', H')$ is lossless in an information-theoretic sense.

Mathematically, each iteration compresses the combined information content of the original object and the random string, denoted as $A+\mathcal{R}$, into a more compact representation comprising the improved object, the truncated random string, and the history log, denoted as $A' + \mathcal{R}' + H'$. The critical observation here is that the entropy (information content) of the original combination $A+\mathcal{R}$ cannot be reduced below its Shannon entropy through lossless compression.

Information-Theoretic Bound

The principle of entropy compression imposes a fundamental limit on the number of iterations an algorithm can perform. Specifically, if the length of the compressed representation $A' + \mathcal{R}' + H'$ grows slower than the length of the original representation $A + \mathcal{R}$, then the algorithm cannot continue indefinitely without violating the entropy bound.

Formally, let $|A+\mathcal{R}|$ denote the total information content of the object and the random string before compression, and let $|A' + \mathcal{R}' + H'|$ denote the information content after compression. According to Shannon's entropy theorem:

$$E[|A' + \mathcal{R}' + H'|] \geq H(A + \mathcal{R}),$$

where $H(\cdot)$ represents the Shannon entropy.

To ensure termination, the algorithm must satisfy the condition:

$$E[|A' + \mathcal{R}' + H'|] < E[|A + \mathcal{R}|].$$

This inequality implies that the algorithm reduces the information content in each iteration, leading to a bounded number of iterations before termination is guaranteed. By leveraging information-theoretic bounds to ensure that the algorithm cannot perpetually make progress without exhausting the available randomness.

With Tao's general ideas set, it is now possible to go deeper and examine Moser's Entropy Compression Argument, starting with some basic information theoretic background.

Information Theoretic Background:

Information Content

[Information Content] The *information content*, also known as *surprisal*, of an event E is defined as

$$I(E) = -\log_2 \Pr(E),$$

where the logarithm is taken to base 2.

The information content quantifies the amount of information gained upon the occurrence of the event E. Intuitively, rarer events (those with lower probability) convey more information when they occur compared to more frequent events.

Entropy

[Entropy] The (Shannon) *entropy* of a discrete random variable X with finite support $\mathcal{X}$ is defined as

$$H(X) = E_{x\sim X}[I(X = x)] = -\sum_{x\in\mathcal{X}} \Pr(X = x) \log_2 \Pr(X = x).$$

Entropy measures the expected amount of information obtained from observing the outcome of the random variable X. It represents the uncertainty inherent in X's possible outcomes. The choice of logarithm base 2 aligns entropy with the concept of information measured in bits, and this definition is deeply connected to coding theory, where entropy quantifies the optimal compression achievable.

Uniquely Decodable Codes

[Uniquely Decodable Codes] For two alphabets (i.e., sets of symbols) Σ_1 and Σ_2, a *code* is a function

$$C : \Sigma_1 \to \Sigma_2^*$$

mapping each symbol of Σ_1 to a string of symbols over Σ_2. The *extension* of C is the natural homomorphism it induces from Σ_1^* to Σ_2^*, defined by mapping each string $s_1 s_2 \dots s_n \in \Sigma_1^*$ to

$$C(s_1)C(s_2)\dots C(s_n)) \in \Sigma_2^*.$$

A code is *uniquely decodable* if its extension is injective, meaning that no two distinct strings in Σ_1^* map to the same string in Σ_2^*.
Uniquely decodable codes ensure that each encoded message can be perfectly recovered without ambiguity.

Shannon's Source Coding Theorem

[Shannon's Source Coding Theorem] Let X be a discrete random variable with finite support $\mathcal{X}$, and let $f : \mathcal{X} \to \{0, 1\}^*$ be a uniquely decodable code. Then, the expected length of the codeword $f(X)$ satisfies

$$E[|f(X)|] \geq H(X).$$

The theorem asserts that any uniquely decodable encoding of a random variable X must have an expected codeword length that is at least the entropy of X. This establishes a fundamental limit on the efficiency of noiseless data compression, ensuring that one cannot, on average, compress the information below its entropy without loss.

Sloan's Useful Example:

Consider a random variable X uniformly distributed over the set $\{0,1\}^r$, denoted as $X \sim U(\{0,1\}^r)$. The entropy of X is

$$H(X) = -\sum_{x\in\{0,1\}^r} \Pr(X=x)\log_2 \Pr(X=x) = -\sum_{x\in\{0,1\}^r} \frac{1}{2^r}\log_2 \frac{1}{2^r} = r \text{ bits.}$$

According to Shannon's Source Coding Theorem, for any uniquely decodable code $f : \{0,1\}^r \to \{0,1\}^*$, the expected length of the codeword satisfies

$$E[|f(X)|] \geq H(X) = r.$$

This implies that r random bits cannot be noiselessly compressed into fewer than r bits in expectation when using a uniquely decodable code.

With this background, it is now possible to formally examine the previously stated entropy compression argumentation by Terance Tao.

Entropy Compression Theorem

[Entropy Compression] Consider an algorithm A that has access to a stream of bits sampled independently and uniformly at random. Suppose that A proceeds through a sequence of steps $t = 0, 1, 2, \ldots$ until it terminates. For any input, the following conditions hold:

1. A can be modified to maintain a bit string L_t that logs its history after each step t, such that the random bits read up to step t can be recovered from L_t. The computational cost or even the computability of this history log modification is irrelevant.

2. The length of L_t satisfies

$$|L_t| \leq \ell_0 + t\Delta\ell$$

after step t.

3. The number of random bits read up to step t is

$$r_0 + t\Delta r.$$

If

$$\Delta\ell < \Delta r,$$

then the step after which A terminates is, in expectation, at most

$$\frac{\ell_0 - r_0}{\Delta r - \Delta\ell}.$$

Proof:

The proof of the Entropy Compression Theorem utilizes source coding to establish a bound on the expected termination step of the algorithm A.

Proof. **1. Fixed Input Analysis:** Consider a fixed input to the algorithm A. For each bit string y of length $r_0 + t\Delta r$, which represents the random bits available up to step t, let $s(y) \leq t$ denote the step at which the algorithm A terminates when its random bit stream is replaced by y. Consequently, $L_{s(y)}(y)$ denotes the final history log, and $z(y)$ represents the suffix of y consisting of the $(t - s(y))\Delta r$ bits unused due to early termination.

2. Unique Decodability: Define a coding function $f : \{0,1\}^{r_0+t\Delta r} \to \{0,1\}^*$ by concatenating the history log and the unused bits:

$$f(y) = L_{s(y)}(y) \circ z(y),$$

where $\circ$ denotes concatenation. The assumption that f is uniquely decodable implies that each

bit string y can be uniquely recovered from $f(y)$. This means that the encoding f must satisfy:

$$|f(y)| \geq H(y),$$

where $H(y)$ is the entropy of the random bit string y.

3. Length Analysis: The length of $f(y)$ satisfies:

$$|f(y)| = |L_{s(y)}(y)| + |z(y)| \leq \ell_0 + s(y)\Delta\ell + (t - s(y))\Delta r.$$

Taking the expectation over all possible random bit strings Y, we obtain:

$$E[|f(Y)|] \leq \ell_0 + E[S]\Delta\ell + (t - E[S])\Delta r,$$

where $S = s(Y)$ is the random variable representing the termination step.

4. Entropy Lower Bound: The entropy of Y, given that Y consists of $r_0 + t\Delta r$ independent and uniformly random bits, is:

$$H(Y) = r_0 + t\Delta r.$$

Since f is uniquely decodable, Shannon's Source Coding Theorem dictates that:

$$E[|f(Y)|] \geq H(Y) = r_0 + t\Delta r.$$

5. Combining Inequalities: Combining the upper and lower bounds on $E[|f(Y)|]$, we obtain:

$$\ell_0 + E[S]\Delta\ell + (t - E[S])\Delta r \geq r_0 + t\Delta r.$$

Simplifying the inequality:

$$\ell_0 + E[S]\Delta\ell \geq r_0.$$

Rearranging terms to solve for $E[S]$:

$$E[S](\Delta r - \Delta\ell) \leq \ell_0 - r_0,$$

$$E[S] \leq \frac{\ell_0 - r_0}{\Delta r - \Delta \ell}.$$

6. Finalizing the Bound: Since $\Delta\ell < \Delta r$, the denominator $\Delta r - \Delta\ell$ is positive, ensuring that $E[S]$ is well-defined and non-negative. This establishes that the expected number of steps before termination is bounded by:

$$E[S] \leq \frac{\ell_0 - r_0}{\Delta r - \Delta \ell}.$$

7. Extending to Infinite Steps: Let T denote the terminating step of the algorithm when provided with an infinite stream of random bits (with the convention that $T = \infty$ if the algorithm fails to halt). Since the bound on $E[S]$ is independent of t, it also applies to:

$$E[T] = \lim_{t \to \infty} E[S].$$

Therefore, the algorithm A is guaranteed to terminate in expectation within the bound specified, provided that the entropy compression condition $\Delta\ell < \Delta r$ is satisfied.

□

Interpretation: The theorem essentially states that if an algorithm is designed such that it logs its history in a compressed manner relative to the amount of randomness it consumes, then the algorithm cannot run indefinitely. This is analogous to an optimal compression scheme where the information content cannot be reduced below its entropy, thereby enforcing a limit on the algorithm's runtime. The compression condition $\Delta\ell < \Delta r$ ensures that with each step, the algorithm is "using up" more randomness than it is "logging," which, by information-theoretic principles, bounds the number of possible steps the algorithm can take before termination.

Analysis of Moser's Resampling Algorithm:

With all the necessary background steadily established, and the developments systematically examined, it is finally possible to state the analysis of Moser's resampling algorithm which is the culmination of it all. For maximal understanding and simplicity, it is general practice to restrict the setting in hand to $k - SAT$ requiring $k - CNF$ formulae to have exactly K distinct

variables in each clause. This is actually better as the $k - SAT$ problem is of fundamental importance in theoretical computer science. Consider the following algorithm:

Algorithm 2 RESAMPLE-2

Require: k-CNF formula ϕ

Ensure: Assignment x such that $\phi(x)$ holds

```
Function SOLVE(φ)
    Randomly assign each x_i to True or False with equal probability
    while there exists an unsatisfied clause C ∈ φ do
        x ← FIX(x, C, φ)
    end while
    return x
Function FIX(x, C, φ)
    Randomly resample each x_i ∈ C
    while there exists an unsatisfied clause D ∈ φ sharing a variable with C do
        x ← FIX(x, D, φ)
    end while
    return x
```

The RESAMPLE2 algorithm operates by initially assigning random truth values to all variables in the k-CNF formula ϕ with the added recursive step of fixing nearby clauses before continuing to other unsatisfied clauses.

Running Time of Resample-2

The following theorem provides an upper bound on the expected running time assuming slightly relaxed symmetric LLL conditions, under a model of computation where sampling a single random bit takes $O(1)$ time.

[Running Time of RESAMPLE2] Let ϕ be a k-CNF formula with n variables and m clauses. Suppose that each clause in ϕ shares variables with at most $\frac{2^k}{C}$ other clauses, where C is an absolute constant. Under these conditions, the RESAMPLE2 algorithm terminates in expected

time

$$\mathcal{O}(n + m \cdot k),$$

Note: Setting $C = \log_2(e)$ aligns the entropy compression conditions precisely with the symmetric Lovász Local Lemma (LLL) conditions.

Proof:

To apply the entropy compression argument, the execution of the RESAMPLE2 algorithm must be partitioned into discrete steps. The initial assignments made by the SOLVE function constitute step $t = 0$, and each subsequent resampling of a clause by the FIX function constitutes an additional step $t = 1, 2, \ldots$. The proof proceeds by demonstrating that the algorithm cannot perform an excessive number of resampling steps without violating information-theoretic bounds.

1. Unique Decodability and Recovery of Random Bits A crucial observation is that only one specific assignment of the relevant k variables can cause a k-CNF clause to be violated, thereby necessitating a resampling step. Given a complete stack trace of an algorithm run, it is possible to reconstruct the sequence of random bits used by the algorithm by working backwards from the final assignment x.

Example: Suppose the last clause C of a 3-CNF formula was $x_1 \vee \neg x_2 \vee x_3$ and was the final clause to be fixed by RESAMPLE2. The final assignments $(x_1, x_2, x_3) = (\text{False}, \text{True}, \text{False})$ imply that the last three bits sampled were $(\text{False}, \text{True}, \text{False})$. This allows the reconstruction of the previous state before the last resampling.

2. History Logging Protocol Using the dependency bound $d \leq \frac{2^k}{C}$, an efficient history logging protocol can be constructed. The history log L_t after t steps consists of the following components:

1. **Variable Assignments:** The initial n bits specifying the current assignment of variables.

2. **Fixed Clauses:** A bitmask of m bits indicating which clauses have been fixed by non-recursive calls from SOLVE.

3. **Recursive Call Logs:** For each recursive call to FIX, an index is logged corresponding to its position within an ordered list of $d+1$ clauses that share variables with the parent clause. This requires

$$\lceil \log_2(d+1) \rceil + \mathcal{O}(1)$$

bits per recursive call, where the $\mathcal{O}(1)$ term accounts for additional control information necessary to manage the recursion stack.

The unique decodability condition ensures that given A', R', and H' (the truncated random string and the history log), the original object A and the remaining random bits can be reconstructed unambiguously.

3. Bounding the History Log Length After t calls to FIX, the total length of the history log L_t is bounded by:

$$|L_t| \leq n + m + t\left(\log_2(d) + \mathcal{O}(1)\right).$$

Here, n accounts for the initial variable assignments, m for the fixed clauses, and $t(\log_2(d) + \mathcal{O}(1))$ for the recursive call logs.

4. Bounding the Random Bits Consumption The algorithm samples random bits in two phases:

1. **Initial Assignments:** n random bits are sampled to assign truth values to the n variables.

2. **Resampling Steps:** For each call to FIX, k random bits are sampled to resample the k variables in the clause.

Thus, the total number of random bits consumed after t steps is:

$$r_t = n + kt.$$

5. Entropy Compression Condition The history log's growth rate per step is $\log_2(d) + \mathcal{O}(1)$, while the randomness consumption rate per step is k. To apply the entropy compression

argument, it must hold that:

$$\log_2(d) + \mathcal{O}(1) < k.$$

Substituting $d \leq \frac{2^k}{C}$ and $C = \log_2(e)$, we have:

$$\log_2\left(\frac{2^k}{C}\right) + \mathcal{O}(1) = k - \log_2(C) + \mathcal{O}(1).$$

Given that $\log_2(C) = \log_2(\log_2(e)) \approx 1$, for sufficiently large k, the condition $\log_2(d) + \mathcal{O}(1) < k$ is satisfied.

6. Bounding the Number of Resampling Steps Applying the Entropy Compression Theorem, if the rate of information added to the history log is less than the rate of randomness consumption, the algorithm cannot perform an infinite number of steps. Specifically, the expected number of resampling steps t is bounded by:

$$t \leq \frac{\ell_0 - r_0}{\Delta r - \Delta \ell},$$

where:

- $\ell_0 = n + m$ is the initial length of the history log.
- $r_0 = n$ is the initial number of random bits.
- $\Delta \ell = \log_2(d) + \mathcal{O}(1)$ is the growth rate of the history log per step.
- $\Delta r = k$ is the growth rate of random bits per step.

Substituting $\Delta \ell < \Delta r$, we obtain:

$$t \leq \frac{n + m - n}{k - \log_2(d) - \mathcal{O}(1)} = \mathcal{O}\left(\frac{m}{k}\right).$$

Given that m and k are typically related through the structure of the k-CNF formula, this simplifies to an expected running time of:

$$\mathcal{O}(m).$$

7. Total Random Bits Sampled The total number of random bits sampled by RESAMPLE2 is:

$$\mathcal{O}\left(n + m \cdot k\right).$$

This bound is achieved by combining the initial n bits and the k bits sampled per resampling step, multiplied by the expected number of steps $t = \mathcal{O}(m)$.

In essence, the entropy compression argument effectively limits the number of resampling steps an algorithm can perform by ensuring that the history log grows at a slower rate than the consumption of random bits. Consequently, RESAMPLE2 is guaranteed to terminate within an expected time of $\mathcal{O}(n + m \cdot k)$, provided that the dependency conditions are satisfied.

A Brief Remark on the Nature and Implication of the Moser Algorithm and Entropy Argument:

The previously examined proof completes the entropy compression based argument that has been the focus of this report. Keeping as to how one can get lost in the technical details, in his lectures, prof. Gregory Valiant highlights an important point about the nature of this algorithm[7]. It is not simply the case that the algorithm uses a lot (k-bits) of randomness; in-fact, it is necessitated that it uses a lot of randomness in-order for the proof to work. If, instead of using k bits of randomness with every step, it only used k 1 bits, then the proof would not even work, resulting in no bounds on runtime.

One curious implication of this fact concerns the efficiency of greedy algorithms. It is well known from years of experimentation performed, including plenty by myself, that randomness based algorithms perform much better than greedy algorithms in satisfiability problems. A "greedy" algorithm, rather than re-randomizing the variables in a given clause, tries to find an assignment to those k variables that satisfies that clause and as many other clauses are possible. This often results in a case where the greedy algorithm makes great progress initially, but then results in stagnation from continual looping. Such a scheme reduces the amount of randomness consumed by the algorithm, and hence the proof from above would result in a

worse bound on the expected runtime (or a possibly infinite bound). This seems to offer one conceptual explanation as to why randomness based methods are better and why one should want to maximise the amount of randomness used by the algorithm.

It is also possible to observe that approaches such as random search, greedy, etc. fall under the category of algorithms called the stochastic local search algorithms. Moser's entropic argument has now inspired a great deal on using similar argumentation on other stochastic local search methods; an intriguing paper to come out on the same is Dimitris Achlioptas and Fotis Iliopoulos's "Random walks that find perfect objects and the lov asz local lemma."[8]. Terance Tao also states how such argumentation can eventually apply to analysis of PDE's with an amount of random initial data. It is also clear to see how a random bit compression based argument would be strongly connected to Kolmogorov Complexity; Lance Fortnow established a proof using the same in his blog page shortly after Moser's talk[9].

In all, Moser and Tardos's algorithmic construction of the LLL provided a significant break through for the analysis of probablistic and randomized algorithms and the result is widely celebrated in theoretical computer science for the same. Furthermore, Moser's entropy based argumentation for the proof of algorithmic LLL too seems to hold much wider implications, there-by inspiring a class of work of its own. This report has stated the necessary background and examined these extremely exciting developments.

References:

1 Paul Erd os and L aszl o Lov asz. Problems and results on 3-chromatic hypergraphs and some related questions.
In Infinite and finite sets (Colloq., Keszthely, 1973; dedicated to P. Erd os on his 60th birthday), Vol. II, pages 609–627. Colloq. Math. Soc. J anos Bolyai, Vol. 10. North-Holland, Amsterdam, 1975.

2 Molloy, M. (1998). The Probabilistic Method. In: Habib, M., McDiarmid, C., Ramirez-Alfonsin, J., Reed, B. (eds) Probabilistic Methods for Algorithmic Discrete Mathematics.

Algorithms and Combinatorics, vol 16. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-662-12788-9$_1$

3 Robin A. Moser. A constructive proof of the Lov asz local lemma. In STOC'09—Proceedings of the
2009 ACM International Symposium on Theory of Computing, pages 343–350. ACM, New York, 2009.

4 Robin A. Moser and G abor Tardos. A constructive proof of the general Lov asz local lemma. J.
ACM, 57(2):Art. 11, 15, 2010.

5 Tao, Terence. "Moser's entropy compression argument", https://terrytao.wordpress.com/2009/08/05/
entropy-compression-argument

6 Neitret, Sloan. "Moser's Algorithm and the Lovász Local Lemma", https://www.cs.cornell.edu/ nieter
algorithm-and-the-lovaz-local-lemma/

7 Valiant, Gregory. CS265/CME309: Randomized Algorithms and Probabilistic Analysis Lecture 12: The Constructive Lovasz Local Lemma (Moser's Entropic Proof).
https://theory.stanford.edu/ valiant/teaching/CS265/lectureNotes/l12.pdf

8 Dimitris Achlioptas and Fotis Iliopoulos. Random walks that find perfect objects and the lov asz
local lemma. Journal of the ACM (JACM), 63(3):22, 2016.

9 Fortnow, Lance. A Kolmogorov Complexity Proof of the Lovász Local Lemma. https://blog.computat
complexity-proof-of-lov.html